



Lab 9: Pointers

CSE 4108

Structured Programming I Lab

Lab Tasks

1. Pointers:

Write a program that declares and initializes (to any value you like) a double, an int, and a char. Next declare and initialize a pointer to each of the three variables. Your program should then print the address of, and value stored in, and the memory size (in bytes) of each of the variables and their corresponding pointers. Here's an example of the desired output:

```
The address of char c is 0x61feff
The address of int i is 0x61fef4
The address of double d is 0x61fee8
The address of char* pc is 0x61fef8
The address of int* pi is 0x61fef0
The address of double* pd is 0x61fee4
```

```
The value of char c is a
The value of int i is 1
The value of double d is 2.500000
The value of char* pc is 6422271
The value of int* pi is 6422260
The value of double* pd is 6422248
```

```
The size of char is 1
The size of int is 4
The size of double is 8
The size of char* is 4
The size of int* is 4
The size of double* is 4
```

Note: Use the "0x%x" format specifier to print the addresses in hexadecimal.

Hint: Use the function `sizeof()` to get the memory size.

2. Swap:

Write a program that contains the following function:

```
void swap (int *x, int *y);
```

The function will swap the values pointed by x and y. The main function will pass addresses of two variables which need to be swapped. And again, remember, a function does what it says. So the function itself should only do the swapping, it SHOULD NOT scan or print anything.

3. Multitasker:

Being inspired by Walter from Scorpion, you decided to write a program that can multi-task. What a wonderful program would that be! To achieve your goal, you'll write a function in C that would take two integers as input and give back the LCM and GCD of those two numbers. Two totally different fruits from a single tree! How incredible! The declaration of the function is like below:

```
void GCD_LCM(int n1, int n2, int *out_gcd, int *out_lcm);
```

Design the input and outputs accordingly. Remember to keep the scanning and printing in main function.

[Hint] Since you require two things back from the function, you need to use pointers.

4. Cyber Shepherds:

Freshmen students of IUT have recently started their Cyber Security training and have made a group, Cyber Shepherds. As part of the training, they've come to know that most of the data breaches/account hacks occur due to use of simple, easy-to-guess passwords. They also learnt some good practices to follow to secure their own accounts. One such practice is that password should contain at least one special character and one digit other than letters.

As an elite member of Cyber Shepherds, you've decided to go further to help your peers. You want to write a program that'll automatically check a given password to determine if it

satisfies the above condition of a good password. Your program will have a function that'll give back whether the password contains any special character and digit. The function will set two pointers based on the mentioned checks.

Your program will output Strong Password! if the password has both a special character and a digit. It'll print Weak Password! otherwise. Password length will not exceed 30 for this program. The declaration of the function is like below:

```
void checkPasswd(char password[], int length, int *hasSpecial,  
                 int *hasDigit);
```

The pointer `hasSpecial` and `hasDigit` will be set to 1 if the password has any special character and digit respectively, or to 0 if it does not have any. For now, consider { @, #, \$, %, =, &, _ } as special characters. Do the scanning and printing in main function.

[Sample Program]

Input 1

Enter password: password123

Output 1

Weak Password!

Input 2

Enter password: FN\$1caHva=t

Output 2

Strong Password!

5. Max Term:

Write a function in C that would take an array of numbers and return a pointer to the maximum number within the input array. The length of the array will also be passed to the function. The declaration of the function is like below.

```
int* getPtrToMax(int arr[], int len);
```

To clarify, if the function is called on, {55, 69, 48, 77, 10}, it will return pointer to the 4th element with value 77. Design the input and output accordingly to test your program. Again, do the scanning and printing in main function.